

■ EXPERIMENT REPORT: STAGE 1.4.2.5.4

Yes — and this output is very significant for our Stage 1.4.2.5.4 experiment.

You got:

```
===== ARCHITECTURE TEXT TEST =====
```

```
Producer          -> FOUND
Event Hubs        -> FOUND
Stream Consumer   -> FOUND
Data Explorer     -> FOUND
But we need to be careful about what this proves.
```

■ Step 1 — What the code was testing

The code was essentially:

```
markdown_text = doc.export_to_markdown()

architecture_words = [
    "Producer",
    "Event Hubs",
    "Stream Consumer",
    "Data Explorer",
]

for word in architecture_words:
    found = word.lower() in markdown_text.lower()

    print(
        f"{word:20} -> "
        f"{'FOUND' if found else 'NOT FOUND'}"
    )
```

The test asks:

"Do these words occur somewhere in the Markdown exported from the DoclingDocument?"

Your answer is:

Yes, all four occur somewhere in the exported Markdown.

But this does not yet prove that Docling extracted them from the architecture image.

■ Step 2 — Why?

Look at our sample PDF.

It contains normal text such as:

An enterprise application publishes events to Azure Event Hubs.

It also contains the architecture diagram.
Therefore:

"Event Hubs"

could appear in the Markdown because it was extracted from the normal PDF text, not because Docling read it from the diagram.

This is the crucial distinction.

■ Step 3 — Let's prove where the text came from

We need to perform a much more precise experiment.
Run this:

```
architecture_words = [
    "Producer",
    "Event Hubs",
    "Stream Consumer",
    "Data Explorer",
]

print("===== DOCLING DOCUMENT ITEMS =====")

for item, level in doc.iterate_items():

    text = getattr(item, "text", "")

    if text:

        for word in architecture_words:

            if word.lower() in text.lower():

                print(
                    f"\nFOUND: {word}"
                )

                print(
                    "Item type:",
                    type(item).__name__
                )

                print(
                    "Text:",
                    repr(text)
                )
```

This will tell us which Docling item contains those words.

■ Step 4 — What I expect

Suppose we get:

```
FOUND: Event Hubs
Item type: TextItem
Text: 'An enterprise application publishes events to Azure Event Hubs.'
```

That tells us:

```
Event Hubs
↓
Normal PDF text
↓
TextItem
```

It does not prove:

```
Architecture image
↓
OCR/VLM
↓
Event Hubs
```

Likewise, if Producer occurs in ordinary text elsewhere, that could explain its presence.

■ Step 5 — Our test PDF has an important complication

This is something I want to make explicit because it affects the validity of our experiment. Our PDF was designed to contain:

```
normal text
table
architecture diagram
formula image
```

Some words appearing in the architecture diagram also appear in the surrounding normal text. Therefore, this test:

```
word in markdown_text
```

is not sufficient to establish image understanding.

This is a classic experimental-design problem:

The test data contains the same vocabulary in multiple places.

So we need to remove that ambiguity.

■ Step 6 — The strongest test

Let's inspect the actual `PictureItem`.

Run:

```
for i, picture in enumerate(doc.pictures, start=1):

    print("\n" + "=" * 70)
    print(f"PICTURE {i}")
    print("=" * 70)

    print(picture.model_dump())
```

We're particularly interested in whether the picture contains an annotation/description.

■ Step 7 — Search specifically for the picture description

Run:

```
for i, picture in enumerate(doc.pictures, start=1):

    print(f"\n===== PICTURE {i} =====")

    annotations = getattr(
        picture,
        "annotations",
        None
    )

    print("Annotations:")
    print(annotations)
```

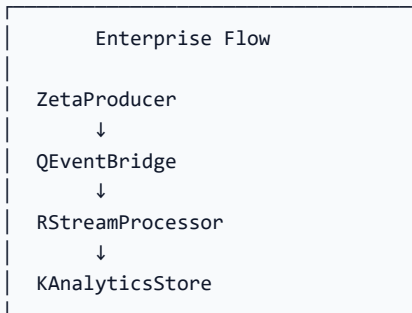
If Level 3 picture understanding has happened, we should see something representing a generated description. Conceptually:

```
PictureItem
|
├── image
|
├── provenance
|
└── annotations
    └── generated description
```

■ Step 8 — There's an even better experiment

We can make our test unambiguous.

Create an architecture diagram containing unique text such as:



Those names should exist only inside the image.

Then test:

```

unique_words = [
    "ZetaProducer",
    "QEventBridge",
    "RStreamProcessor",
    "KAnalyticsStore",
]
  
```

Now:

```

markdown_text = doc.export_to_markdown()

for word in unique_words:

    print(
        f"{word:20} -> "
        f"{'FOUND' if word.lower() in markdown_text.lower() else 'NOT FOUND'}"
    )
  
```

This eliminates the ambiguity.

■ Step 9 — Why this matters

Imagine the result is:

```

ZetaProducer      -> FOUND
QEventBridge      -> FOUND
RStreamProcessor  -> FOUND
KAnalyticsStore    -> FOUND
  
```

Now we have much stronger evidence that some image-processing mechanism has extracted the information.

But if we get:

```

ZetaProducer      -> NOT FOUND
QEventBridge      -> NOT FOUND
RStreamProcessor  -> NOT FOUND
KAnalyticsStore    -> NOT FOUND
  
```

while:

```
len(doc.pictures)
```

is still:

```
1
```

then we have demonstrated:

```
Picture detection
Picture extraction
Text extraction
```

And if the picture annotation contains a meaningful description, then:

```
Picture detection
Picture extraction
Semantic understanding
Exact OCR           possibly
```

That's a very valuable result.

■ Step 10 — So what does YOUR current result mean?

Your current result:

```
Producer      -> FOUND
Event Hubs    -> FOUND
Stream Consumer -> FOUND
Data Explorer -> FOUND
```

means only:

All four strings exist somewhere in Docling's Markdown output.

It does not yet prove:

"Docling read those four strings from the architecture diagram."

Therefore, don't mark Level 3 as completed yet.

We're actually doing the right thing by questioning the result.

■ Current status

Stage 1.4.2.5.4

Docling Table / Image / Formula Understanding

Level 1 — Picture Detection

☐ Demonstrated

Level 2 — Picture Extraction

☐ Demonstrated

Level 3 — Image Content Understanding & Semantic Extraction

☐ Investigating

■ Next experiment

First run the `doc.iterate_items()` diagnostic above.

Paste its output here. From that output, we'll determine whether those FOUND values came from normal TextItems or from the picture-understanding enrichment. That will give us a scientifically valid answer rather than assuming that FOUND means OCR/VLM succeeded.